Table des matières

Labo 11 OLTP — Transactions et concurrence 1
Le terrain, en une minute 1
Partie 1 — Le tout ou rien 2
Partie 2 — Les anomalies de lecture 4
Partie 3 — La mise à jour perdue 6
Partie 4 — L'anomalie d'écriture, et le niveau sérialisable 7
Partie 5 — Blocage et interblocage 8
Partie 6 — Le MVCC, vu de l'intérieur 10
Partie 7 — Rendre un chargement rejouable 11
Partie 8 — Sur l'entrepôt 13
Partie 9 — Ce que le contraste vous a montré 15
Nettoyage 16
Et ensuite ? 16

Labo 11 OLTP — Transactions et concurrence

Laboratoire du Cours 11, voie OLTP · PostgreSQL + AdventureWorks OLTP · Formation SQL avancé

Ce laboratoire se fait à deux sessions ouvertes côte à côte. C'est la seule façon de voir une anomalie de concurrence : elle n'existe pas dans une session isolée, et aucune lecture de documentation ne remplace le moment où l'on voit une requête rester suspendue parce qu'une autre fenêtre tient un verrou. Comptez une heure et demie.

Les deux voies — Ce labo n'a pas de jumeau, et le suffixe `_OLTP` dit sur quel moteur on travaille. La partie 8 traverse quand même sur l'entrepôt, où le problème se pose tout autrement : il n'y a qu'un écrivain à la fois, et le chapitre des niveaux d'isolation y devient sans objet.

Comment ouvrir deux sessions — Dans `psql`, ouvrez deux terminaux et lancez la commande de connexion dans chacun. Dans pgAdmin, ouvrez deux fenêtres d'outil de requête sur la même base ; attention, deux onglets de la même fenêtre partagent parfois la connexion, donc vérifiez avec la requête de contrôle ci-dessous. Dans DBeaver, dupliquez la connexion par le menu contextuel, ou ouvrez un éditeur SQL avec une connexion séparée.

Le terrain, en une minute

© 2026 iAlexMG. Tous droits réservés.

On travaille sur des tables jetables, parce qu'on va les corrompre exprès. Leur construction (clés primaires, `CHECK`, types et identité) est celle du Cours 03, sections 2 et 3.

```sql
CREATE SCHEMA IF NOT EXISTS labo11;
SET search_path TO labo11, public;

CREATE TABLE labo11.compte (
    id      int PRIMARY KEY,
    nom     text NOT NULL,
    solde   numeric(12,2) NOT NULL CHECK (solde >= 0)
);
INSERT INTO labo11.compte VALUES (1, 'Beaupre', 500.00), (2, 'Cormier', 500.00);

CREATE TABLE labo11.garde (
    medecin     text PRIMARY KEY,
    en_service  boolean NOT NULL
);
INSERT INTO labo11.garde VALUES ('Tremblay', true), ('Gagnon', true);

CREATE TABLE labo11.evenement (
    id     bigserial PRIMARY KEY,
    jour   date NOT NULL,
    source text NOT NULL,
    montant numeric(12,2) NOT NULL
);
```

Vérifiez maintenant que vos deux fenêtres sont bien deux sessions distinctes. Lancez ceci dans chacune :

```sql
SELECT pg_backend_pid() AS ma_session, current_setting('transaction_isolation') AS isolation;
```

Attendu : deux numéros différents, et `read committed` des deux côtés. Si les deux numéros sont identiques, votre outil partage une connexion et rien de ce qui suit ne fonctionnera.

Dans tout ce qui suit, la fenêtre A est appelée session A et la fenêtre B session B. Exécutez les blocs dans l'ordre indiqué, un à la fois, en alternant.

Partie 1 — Le tout ou rien

1.1 L'annulation

Théorie : `Theorie_11_SQL.md`, sections 1 et 3.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Session A
BEGIN;
UPDATE labo11.compte SET solde = solde - 100 WHERE id = 1;
SELECT * FROM labo11.compte;     -- A voit 400 et 500

-- Session B, pendant que A est encore ouverte
SELECT * FROM labo11.compte;     -- B voit 500 et 500
```

B ne voit pas le travail non validé de A. C'est l'isolation, et c'est la garantie que PostgreSQL offre toujours, à tous les niveaux : la lecture sale n'existe pas ici.

```sql
-- Session A
ROLLBACK;
SELECT * FROM labo11.compte;     -- retour à 500 et 500
```

1.2 La contrainte qui sauve, et le point de reprise

Théorie : `Theorie_11_SQL.md`, section 3.

```sql
-- Session A
BEGIN;
UPDATE labo11.compte SET solde = solde - 100 WHERE id = 1;
SAVEPOINT apres_debit;
UPDATE labo11.compte SET solde = solde - 10000 WHERE id = 2;   -- viole le CHECK
```

Attendu : une erreur de contrainte. Constatez maintenant l'effet secondaire que beaucoup de gens découvrent en production :

```sql
-- Session A, toujours dans la transaction
SELECT * FROM labo11.compte;   -- ERREUR : la transaction est en échec
```

Une transaction PostgreSQL en échec refuse toute commande jusqu'à ce qu'on l'annule ou qu'on revienne à un point de reprise. C'est le point de reprise qui sauve la mise :

```sql
-- Session A
ROLLBACK TO SAVEPOINT apres_debit;
SELECT * FROM labo11.compte;   -- fonctionne de nouveau, le débit de 100 est conservé
UPDATE labo11.compte SET solde = solde + 100 WHERE id = 2;
COMMIT;
SELECT * FROM labo11.compte;   -- 400 et 600
```

1.3 Le DDL transactionnel, la particularité de PostgreSQL

Théorie : `Theorie_11_SQL.md`, sections 3 et 6. Théorie : `Theorie_03_SQL.md`, sections 2 et 8.

```sql
-- Session A
BEGIN;
```


© 2026 iAlexMG. Tous droits réservés.

```
CREATE TABLE labo11.jetable (x int);
ALTER TABLE labo11.compte ADD COLUMN devise text DEFAULT 'CAD';
SELECT * FROM labo11.jetable;      -- la table existe, dans cette transaction

-- Session B
SELECT * FROM labo11.jetable;      -- ERREUR : la relation n'existe pas
```

La table neuve est invisible hors de la transaction qui l'a créée. La table modifiée, elle, est pire qu'invisible :

```
-- Session B
SELECT * FROM labo11.compte;       -- reste suspendue
```

Observez. Aucune erreur, aucun message : la session B attend. L'`ALTER TABLE` de A tient sur `compte` un verrou d'accès exclusif jusqu'à la fin de sa transaction, `AccessExclusiveLock` dans `pg_locks`, mesuré, et ce verrou arrête aussi les lectures.

```
-- Session A
ROLLBACK;
SELECT * FROM labo11.jetable;      -- ERREUR : elle n'a jamais existé
SELECT * FROM labo11.compte;       -- pas de colonne devise
```

La session B repart au moment du `ROLLBACK`, avec les deux comptes, et la colonne `devise` a disparu elle aussi. Mesuré à la rédaction, en gardant la transaction de A ouverte six secondes : B a attendu 6,0 secondes, et reçu sa réponse à l'instant du `ROLLBACK`.

Le DDL est validé d'office sur Oracle, sur MySQL et sur Snowflake, d'après leur documentation, mais pas sur nos deux terrains : PostgreSQL comme l'entrepôt savent annuler une commande de structure, et cela change la façon dont on écrit une migration de schéma, qui peut devenir atomique.

Piège — Cette possibilité a une contrepartie, et vous venez de la voir. Une transaction qui contient du DDL prend des verrous de table et les garde jusqu'à sa fin. Une migration lancée en pleine journée dans une longue transaction fera attendre tout ce qui touche à ces tables, lectures comprises, sans que rien n'échoue. La partie 5 montre comment trouver la session responsable.

Partie 2 — Les anomalies de lecture

2.1 La lecture non répétable

Théorie : `Theorie_11_SQL.md`, section 4.

```
-- Session A
BEGIN;
SELECT solde FROM labo11.compte WHERE id = 1;    -- notez la valeur
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- Session B
UPDATE labo11.compte SET solde = solde + 250 WHERE id = 1;
COMMIT;   -- ou rien, si votre client valide automatiquement

-- Session A, toujours dans la même transaction
SELECT solde FROM labo11.compte WHERE id = 1;    -- la valeur a changé
COMMIT;
```

Deux lectures de la même ligne, dans la même transaction, donnent deux résultats. C'est la lecture non répétable, et c'est le comportement normal du niveau par défaut. Imaginez un rapport qui calcule un total dans une requête et son détail dans une autre.

2.2 La ligne fantôme

Théorie : `Theorie_11_SQL.md`, section 4. Théorie : `Theorie_02_SQL.md`, section 6.

```sql
-- Session A
BEGIN;
SELECT COUNT(*) FROM labo11.evenement WHERE jour = CURRENT_DATE;

-- Session B
INSERT INTO labo11.evenement (jour, source, montant)
VALUES (CURRENT_DATE, 'flux_b', 42.00);

-- Session A
SELECT COUNT(*) FROM labo11.evenement WHERE jour = CURRENT_DATE;   -- une ligne de plus
COMMIT;
```

La ligne n'a pas changé, elle est apparue. C'est le fantôme.

2.3 La lecture répétable règle les deux

Théorie : `Theorie_11_SQL.md`, section 5.

```sql
-- Session A
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT COUNT(*) FROM labo11.evenement WHERE jour = CURRENT_DATE;
SELECT solde FROM labo11.compte WHERE id = 1;

-- Session B
INSERT INTO labo11.evenement (jour, source, montant) VALUES (CURRENT_DATE, 'flux_b', 7.00);
UPDATE labo11.compte SET solde = solde + 1 WHERE id = 1;

-- Session A
SELECT COUNT(*) FROM labo11.evenement WHERE jour = CURRENT_DATE;   -- inchangé
SELECT solde FROM labo11.compte WHERE id = 1;                      -- inchangé
```


© 2026 iAlexMG. Tous droits réservés.

```
COMMIT;
SELECT solde FROM labo11.compte WHERE id = 1;                    -- la nouvelle valeur apparaît
```

Rien n'a bougé pendant toute la transaction, alors que la base a bel et bien changé. La session A travaillait sur un instantané pris à l'ouverture. Notez au passage que PostgreSQL empêche ici les lignes fantômes, ce que la norme n'exige pas à ce niveau.

À retenir — C'est le mode dans lequel un rapport multi-requêtes devrait tourner. Deux lignes de plus au début du script, et le total et le détail ne peuvent plus se contredire.

Partie 3 — La mise à jour perdue

3.1 La perdre

Théorie : `Theorie_11_SQL.md`, section 4.

```
-- Session A
BEGIN;
SELECT solde FROM labo11.compte WHERE id = 1;    -- disons 651

-- Session B
BEGIN;
SELECT solde FROM labo11.compte WHERE id = 1;    -- 651 aussi

-- Session A : retrait de 30, calculé par l'application
UPDATE labo11.compte SET solde = 621 WHERE id = 1;
COMMIT;

-- Session B : retrait de 50, calculé sur la valeur qu'elle avait lue
UPDATE labo11.compte SET solde = 601 WHERE id = 1;
COMMIT;

SELECT solde FROM labo11.compte WHERE id = 1;
```

Attendu : 601. Les deux retraits ont eu lieu, un seul se voit. Aucune erreur, aucun verrou en conflit, et trente dollars ont disparu. C'est l'anomalie la plus coûteuse du module, et la plus facile à produire.

3.2 Les deux corrections

Théorie : `Theorie_11_SQL.md`, sections 5 et 6.

```
-- Correction 1 : laisser le moteur calculer.
UPDATE labo11.compte SET solde = solde - 30 WHERE id = 1;
```


© 2026 iAlexMG. Tous droits réservés.

L'écriture est atomique : le moteur lit et écrit sous le même verrou de ligne, et la seconde transaction attendra puis recalculera à partir de la valeur à jour. C'est la bonne réponse chaque fois que le nouveau montant se déduit de l'ancien.

```
-- Correction 2 : verrouiller à la lecture, quand l'application doit décider.
-- Session A
BEGIN;
SELECT solde FROM labo11.compte WHERE id = 1 FOR UPDATE;

-- Session B
BEGIN;
SELECT solde FROM labo11.compte WHERE id = 1 FOR UPDATE;   -- BLOQUE, et c'est le but

-- Session A
UPDATE labo11.compte SET solde = solde - 30 WHERE id = 1;
COMMIT;    -- B se débloque et lit la valeur à jour

-- Session B
COMMIT;    -- B referme sa transaction avant la partie 4
```

`FOR UPDATE` pose dès la lecture le verrou que l'écriture aurait posé. C'est la réponse quand le calcul se fait hors de la base, dans du code applicatif qui doit vérifier une règle avant d'écrire.

Partie 4 — L'anomalie d'écriture, et le niveau sérialisable

4.1 La produire en lecture répétable

Théorie : `Theorie_11_SQL.md`, section 4.

La règle métier : il doit rester au moins un médecin de garde. Les deux sessions la vérifient honnêtement.

```
-- Session A
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT COUNT(*) FROM labo11.garde WHERE en_service;    -- 2, donc je peux me retirer

-- Session B
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT COUNT(*) FROM labo11.garde WHERE en_service;    -- 2 aussi

-- Session A
UPDATE labo11.garde SET en_service = false WHERE medecin = 'Tremblay';
COMMIT;
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- Session B
UPDATE labo11.garde SET en_service = false WHERE medecin = 'Gagnon';
COMMIT;

SELECT * FROM labo11.garde;
```

Attendu : personne de garde. Les deux transactions ont respecté la règle, les deux ont validé, et la règle est violée. Aucune contrainte de la base ne pouvait l'attraper, parce qu'elle porte sur une propriété collective. Notez que les deux ont écrit sur des lignes différentes : il n'y a jamais eu de conflit de verrou.

4.2 Le niveau sérialisable la refuse

Théorie : `Theorie_11_SQL.md`, section 5.

Remettez tout le monde de garde, puis recommencez d'un cran plus haut.

```sql
UPDATE labo11.garde SET en_service = true;

-- Session A
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM labo11.garde WHERE en_service;
UPDATE labo11.garde SET en_service = false WHERE medecin = 'Tremblay';

-- Session B
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM labo11.garde WHERE en_service;
UPDATE labo11.garde SET en_service = false WHERE medecin = 'Gagnon';

-- Session A
COMMIT;    -- passe

-- Session B
COMMIT;    -- ERREUR de sérialisation
```

Attendu : la session B est annulée avec un message qui parle de dépendances lecture-écriture. Le moteur a détecté que l'entrelacement des deux transactions ne correspond à aucun ordre séquentiel possible, et il en a sacrifié une.

Retenez la forme de la protection. Le moteur n'a rien bloqué, il a laissé les deux travailler, et il a annulé au moment de valider. Une application qui emploie ce niveau doit donc savoir rejouer la transaction annulée, sinon elle a simplement remplacé une donnée fausse par une erreur devant l'utilisateur.

Partie 5 — Blocage et interblocage

5.1 Un blocage ordinaire, et comment le lire

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_11_SQL.md`, section 6. Théorie : `Theorie_02_SQL.md`, section 5.

```
-- Session A
BEGIN;
UPDATE labo11.compte SET solde = solde + 1 WHERE id = 1;
-- on ne valide pas

-- Session B
UPDATE labo11.compte SET solde = solde + 1 WHERE id = 1;   -- reste suspendue

-- Session C, ou la session A dans un autre onglet : qui bloque qui ?
SELECT pid,
       pg_blocking_pids(pid) AS bloque_par,
       state,
       left(query, 60)       AS requete
FROM   pg_stat_activity
WHERE  cardinality(pg_blocking_pids(pid)) > 0;
```

`pg_blocking_pids(pid)` rend la liste des sessions qui bloquent `pid`, `cardinality` compte les éléments de cette liste, et `left(query, 60)` garde les 60 premiers caractères de la requête, comme `SUBSTRING(query, 1, 60)`. Vous tenez là la requête la plus utile du module. Elle répond à « pourquoi l'application est-elle figée » en une ligne, et elle donne le numéro de la session responsable.

```
-- Session A
COMMIT;   -- B repart aussitôt
```

Un blocage n'est pas une erreur. B attendait, indéfiniment s'il l'avait fallu, et rien ne se serait plaint.

5.2 Un interblocage, provoqué exprès

Théorie : `Theorie_11_SQL.md`, section 6.

```
-- Session A
BEGIN;
UPDATE labo11.compte SET solde = solde + 1 WHERE id = 1;

-- Session B
BEGIN;
UPDATE labo11.compte SET solde = solde + 1 WHERE id = 2;

-- Session A : demande la ligne que B tient
UPDATE labo11.compte SET solde = solde + 1 WHERE id = 2;    -- suspendue

-- Session B : demande la ligne que A tient
UPDATE labo11.compte SET solde = solde + 1 WHERE id = 1;    -- interblocage
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : au bout d'une seconde environ, l'une des deux sessions reçoit une erreur d'interblocage, avec le détail des processus et des verrous en cause, et l'autre repart. Le délai vient du paramètre qui décide au bout de combien de temps le moteur va chercher un cycle plutôt que de continuer d'attendre.

Regardez le message d'erreur en entier : il nomme les deux processus, les deux verrous, et la commande de chacun. C'est tout ce qu'il faut pour trouver la cause dans le code.

Annulez la transaction survivante, puis retenez la règle : les deux sessions ont pris les mêmes verrous dans l'ordre inverse. Le même scénario avec les deux mises à jour dans l'ordre 1 puis 2 des deux côtés ne produit qu'un blocage, jamais un interblocage.

```
-- Dans chaque session : annule la survivante, et sort la victime de son échec
ROLLBACK;
```

Partie 6 — Le MVCC, vu de l'intérieur

6.1 Une ligne mise à jour change de place

Théorie : `Theorie_11_SQL.md`, section 7.

```
SELECT ctid, xmin, xmax, id, solde FROM labo11.compte ORDER BY id;
UPDATE labo11.compte SET solde = solde WHERE id = 1;    -- la valeur ne change pas
SELECT ctid, xmin, xmax, id, solde FROM labo11.compte ORDER BY id;
```

Le `ctid` de la ligne 1 a changé, et son `xmin` aussi. La ligne n'a pas été modifiée : une nouvelle version a été écrite ailleurs, et l'ancienne a été marquée comme périmée. Une mise à jour qui ne change rien coûte donc autant qu'une vraie.

6.2 Le gonflement, mesuré

Théorie : `Theorie_11_SQL.md`, section 7.

```
CREATE TABLE labo11.gonflee AS
SELECT g AS id, repeat('x', 200) AS charge FROM generate_series(1, 100000) AS g;
ANALYZE labo11.gonflee;

SELECT pg_size_pretty(pg_relation_size('labo11.gonflee')) AS taille_initiale;

-- Trois mises à jour complètes, qui ne changent aucune valeur.
UPDATE labo11.gonflee SET charge = charge;
UPDATE labo11.gonflee SET charge = charge;
UPDATE labo11.gonflee SET charge = charge;

SELECT pg_size_pretty(pg_relation_size('labo11.gonflee')) AS apres_trois_updates;
```


© 2026 iAlexMG. Tous droits réservés.

```
SELECT n_live_tup, n_dead_tup, last_autovacuum
FROM   pg_stat_user_tables
WHERE  relname = 'gonflee';
```

La table a grossi de plusieurs fois sa taille sans qu'une seule ligne ait changé de valeur ni de nombre. Ce sont les versions périmées. Si la taille a moins augmenté que prévu, regardez la colonne `last_autovacuum` : le démon de maintenance est peut-être passé entre deux commandes et a déjà rendu une partie de l'espace réutilisable, ce qui est précisément son travail.

```
VACUUM (VERBOSE) labo11.gonflee;
SELECT pg_size_pretty(pg_relation_size('labo11.gonflee')) AS apres_vacuum;
```

Un `VACUUM` ordinaire rend l'espace réutilisable sans rendre le fichier au système, donc la taille ne baisse pas forcément. Un `VACUUM FULL` réécrit la table et la rend compacte, au prix d'un verrou exclusif qui bloque tout le monde. C'est la raison pour laquelle on ne le lance pas en pleine journée.

6.3 Le lien avec le Cours 10

Théorie : `Theorie_11_SQL.md`, section 7.

```
CREATE INDEX idx_gonflee ON labo11.gonflee (id);
VACUUM ANALYZE labo11.gonflee;
UPDATE labo11.gonflee SET charge = charge WHERE id <= 30000;

EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo11.gonflee WHERE id BETWEEN 1 AND 20000;
-- Regardez la ligne « Heap Fetches ».

VACUUM labo11.gonflee;
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo11.gonflee WHERE id BETWEEN 1 AND 20000;
-- « Heap Fetches » a chuté.
```

Voilà l'explication que le Cours 10 avait laissée en suspens. Un index ne porte pas l'information de visibilité, donc le moteur doit ouvrir la table pour savoir si une ligne trouvée dans l'index est visible pour vous. Il n'en est dispensé que sur les pages que le nettoyage a marquées comme entièrement visibles. Le parcours d'index seul dépend donc du nettoyage, ce qui n'a aucun sens tant qu'on n'a pas vu le MVCC.

Partie 7 — Rendre un chargement rejouable

C'est la partie qui justifie le module pour un ingénieur analytique.

7.1 Le chargement naïf, et sa fenêtre

Théorie : `Theorie_11_SQL.md`, section 9. Théorie : `Theorie_03_SQL.md`, section 5.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Session A : le chargement, façon script du dimanche.
DELETE FROM labo11.evenement WHERE jour = CURRENT_DATE;
-- ... ici, une pause de quelques secondes le temps que B lise ...
INSERT INTO labo11.evenement (jour, source, montant)
SELECT CURRENT_DATE, 'flux_a', g * 10 FROM generate_series(1, 5) AS g;

-- Session B, à lancer entre les deux commandes de A
SELECT COUNT(*), COALESCE(SUM(montant), 0) FROM labo11.evenement WHERE jour = CURRENT_DATE;
```

Attendu : zéro. Le rapport n'a pas échoué, il n'a rien signalé, il a affiché zéro. C'est exactement le scénario décrit à la section 1 de la théorie.

7.2 Le même chargement, dans une transaction

Théorie : `Theorie_11_SQL.md`, sections 1 et 9.

```sql
-- Session A
BEGIN;
DELETE FROM labo11.evenement WHERE jour = CURRENT_DATE;
-- pause
INSERT INTO labo11.evenement (jour, source, montant)
SELECT CURRENT_DATE, 'flux_a', g * 10 FROM generate_series(1, 5) AS g;

-- Session B, pendant la pause
SELECT COUNT(*), COALESCE(SUM(montant), 0) FROM labo11.evenement WHERE jour = CURRENT_DATE;
```

Attendu : l'état d'avant, complet et cohérent. B ne voit jamais la table amputée.

```sql
-- Session A
COMMIT;
```

Relancez le bloc de A deux fois de suite et vérifiez que le compte reste à cinq. Le chargement est idempotent : la suppression de la tranche puis sa réinsertion laissent le même état, qu'on le lance une fois ou dix.

7.3 La fusion par clé

Théorie : `Theorie_11_SQL.md`, section 9. Théorie : `Theorie_03_SQL.md`, section 6.

```sql
CREATE TABLE labo11.cible (cle int PRIMARY KEY, valeur numeric, maj timestamptz);
CREATE TABLE labo11.source (cle int, valeur numeric);
INSERT INTO labo11.source VALUES (1, 10), (2, 20), (3, 30);

-- Le contrôle qui fait partie du chargement, pas des bonnes intentions.
SELECT cle, COUNT(*) FROM labo11.source GROUP BY cle HAVING COUNT(*) > 1;

MERGE INTO labo11.cible AS c
```


© 2026 iAlexMG. Tous droits réservés.

```
USING labo11.source AS s ON c.cle = s.cle
WHEN MATCHED THEN UPDATE SET valeur = s.valeur, maj = now()
WHEN NOT MATCHED THEN INSERT (cle, valeur, maj) VALUES (s.cle, s.valeur, now());

SELECT * FROM labo11.cible ORDER BY cle;
```

Relancez le `MERGE` : le contenu ne change pas, seule la colonne `maj` avance. Ajoutez maintenant un doublon dans la source et relancez :

```
INSERT INTO labo11.source VALUES (2, 999);
MERGE INTO labo11.cible AS c
USING labo11.source AS s ON c.cle = s.cle
WHEN MATCHED THEN UPDATE SET valeur = s.valeur, maj = now()
WHEN NOT MATCHED THEN INSERT (cle, valeur, maj) VALUES (s.cle, s.valeur, now());
```

Attendu : une erreur signalant qu'une ligne cible serait mise à jour deux fois. Le moteur refuse plutôt que de choisir au hasard, et c'est la bonne nouvelle. Sur d'autres moteurs, la même situation produit un résultat non déterministe, et c'est la mauvaise. Dans les deux cas, la conclusion est la même : le contrôle d'unicité de la source appartient au chargement.

Portabilité — `MERGE` existe sur PostgreSQL depuis sa version 15, sur l'entrepôt, et sur Snowflake d'après sa documentation. Sur une version plus ancienne de PostgreSQL, l'équivalent est `INSERT … ON CONFLICT (cle) DO UPDATE SET …`, qui rend le même service avec une syntaxe différente et qui reste souvent préférable pour un simple ajout avec écrasement.

Partie 8 — Sur l'entrepôt

Passez sur l'entrepôt, dans la ligne de commande DuckDB.

```
USE dbo;

CREATE OR REPLACE TABLE LABO11_CIBLE AS
SELECT SalesOrderNumber, SalesOrderLineNumber, SalesAmount
FROM   dbo.FactInternetSales LIMIT 1000;

SELECT COUNT(*), SUM(SalesAmount) FROM LABO11_CIBLE;
```

8.1 La transaction existe aussi

Théorie : `Theorie_11_SQL.md`, section 8.

```
BEGIN;
DELETE FROM LABO11_CIBLE;
SELECT COUNT(*) FROM LABO11_CIBLE;    -- 0, dans cette transaction
```


© 2026 iAlexMG. Tous droits réservés.

```
ROLLBACK;
SELECT COUNT(*) FROM LABO11_CIBLE;     -- 1000, tout est revenu
```

Attendu : 0 puis 1000. Rien de surprenant, et c'est le but : les transactions font ici exactement ce qu'elles font sur le transactionnel.

8.2 Le DDL s'annule ici aussi

Théorie : `Theorie_11_SQL.md`, section 8.

```
BEGIN;
CREATE OR REPLACE TABLE LABO11_JETABLE (x INTEGER);
ROLLBACK;
SELECT COUNT(*) AS existe FROM duckdb_tables() WHERE table_name = 'LABO11_JETABLE';
```

Attendu : zéro, la table n'a jamais existé. C'est le comportement mesuré au Cours 03, section 8, et il place nos deux terrains du même côté, celui des moteurs qui savent annuler une commande de structure. La plupart des entrepôts font l'inverse et valident la transaction en cours au moment du `CREATE` : ne supposez donc pas ce confort acquis ailleurs.

8.3 Un seul écrivain à la fois

Théorie : `Theorie_11_SQL.md`, section 8.

Voici ce qui remplace, sur un moteur intégré, tout le chapitre des niveaux d'isolation. Ouvrez une **seconde** ligne de commande, et lancez-y une écriture qui dure une poignée de secondes :

```
-- Dans la seconde session.
USE dbo;
CREATE OR REPLACE TABLE zz_longue AS
SELECT f.*, g.n
FROM   dbo.FactInternetSales AS f, (SELECT UNNEST(range(1, 120)) AS n) AS g;
```

Pendant qu'elle tourne, essayez d'ouvrir l'entrepôt depuis une troisième session, même en lecture seule, par `duckdb -readonly adventureworksdw.duckdb`.

Attendu : l'ouverture échoue tout de suite, avec un message qui nomme le coupable.

```
IO Error: Cannot open file "adventureworksdw.duckdb": Le processus ne peut pas accéder au fichier
car ce fichier est utilisé par un autre processus.
File is already open in … duckdb.exe (PID 14064)
```

Mesurez ce que cela veut dire, parce que c'est l'écart le plus profond de tout le module. Il n'y a ici ni verrou de ligne, ni MVCC, ni niveau d'isolation à choisir : il y a un processus qui détient le fichier, et tous les autres attendent. Les parties 3 à 6 de ce laboratoire, celles où deux sessions se disputent une ligne, n'ont tout simplement pas d'équivalent.

© 2026 iAlexMG. Tous droits réservés.

Le message est net, ce qui est une chance : ce conflit-là se diagnostique en une seconde, là où un interblocage sur le transactionnel demande de lire une vue système. Et la contrainte est acceptable parce qu'elle correspond à l'usage : on analyse depuis un entrepôt local, on ne fait pas tourner vingt applications dessus.

Piège — C'est aussi la cause d'erreur la plus fréquente quand on enchaîne un labo et un script Python sur le même fichier : fermez la ligne de commande avant de lancer le script, ou l'inverse.

8.4 Ce que l'entrepôt local n'a pas

Théorie : `Theorie_11_SQL.md`, section 8.

Les grandes plateformes offrent un filet que notre entrepôt n'a pas, le voyage dans le temps : interroger une table telle qu'elle était il y a une heure, ou avant une commande précise, et récupérer par `UNDROP TABLE` une table supprimée par erreur. C'est décrit dans la documentation de Snowflake, et vous ne pouvez pas l'essayer ici.

L'équivalent local est plus rustique et tout aussi efficace : avant une opération risquée, on copie le fichier de l'entrepôt. Vingt-cinq mégaoctets, une seconde, et le filet est posé.

```
-- Depuis la ligne de commande, hors SQL : une copie du fichier suffit.
-- Sur PowerShell : Copy-Item adventureworksdw.duckdb adventureworksdw_avant.duckdb
```

Dans les deux cas, le filet ne remplace pas la transaction : il répare après coup, sur décision humaine, alors que la transaction empêche l'état intermédiaire d'exister. Un rapport lancé pendant les trois secondes où la table était amputée aura quand même donné un chiffre faux, et aucune copie ne le rattrapera.

8.5 Nettoyage de l'entrepôt

```
DROP TABLE IF EXISTS LABO11_CIBLE;
DROP TABLE IF EXISTS LABO11_JETABLE;
DROP TABLE IF EXISTS zz_longue;
```

Partie 9 — Ce que le contraste vous a montré

Théorie : `Theorie_11_SQL.md`, sections 8 et 9.

Sujet	PostgreSQL, ce labo	L'entrepôt, partie 8
Niveaux d'isolation	quatre demandés, trois distincts en pratique	aucun : un seul écrivain, rien à régler
Lecture sale	impossible, à tous les niveaux	sans objet, personne ne lit pendant qu'un autre écrit

© 2026 iAlexMG. Tous droits réservés.

Sujet	PostgreSQL, ce labo	L'entrepôt, partie 8
Le DDL dans une transaction	annulable, particularité rare et précieuse	annulable aussi, mesuré
Le conflit typique	deux sessions sur la même ligne	deux processus sur le même fichier, et le second est refusé
Comment le conflit se voit	un blocage, à diagnostiquer dans une vue système	une erreur immédiate qui nomme le processus et son PID
Voir un état passé	rien, sauf une sauvegarde	rien non plus : on copie le fichier avant l'opération risquée
Récupérer une table supprimée	une restauration	la copie du fichier, ou rien
Le coût du modèle	versions périmées, nettoyage, gonflement	un seul écrivain à la fois, assumé
Rendre un chargement rejouable	transaction plus suppression par tranche, ou `MERGE`	exactement les mêmes motifs

La leçon de fond tient en une phrase. Le transactionnel se protège des conflits ligne à ligne, et l'entrepôt s'en protège en n'en ayant presque jamais ; mais les deux ont besoin qu'un chargement soit atomique et rejouable, et c'est la seule partie du module qui vous suivra partout.

Une remarque sur la première ligne, qui pourrait se lire comme une pauvreté. Les quatre niveaux d'isolation que vous venez d'éprouver ne manquent pas à l'entrepôt : ils n'y ont pas d'objet, faute de deux écrivains simultanés à départager. Les grandes plateformes, elles, acceptent bien plusieurs écrivains et retrouvent alors le besoin d'un niveau d'isolation, un seul, la lecture validée, avec des fusions concurrentes qui se sérialisent, d'après la documentation de Snowflake. Le chapitre que vous venez de faire n'est donc pas propre au transactionnel : il est propre aux moteurs qui acceptent plusieurs écrivains.

Nettoyage

```
DROP SCHEMA labo11 CASCADE;
```

Et ensuite ?

Vous savez maintenant pourquoi un rapport peut se contredire lui-même, pourquoi une application se fige sans erreur, et pourquoi un chargement doit pouvoir être relancé sans dommage. Vous avez aussi récupéré l'explication que le Cours 10 devait à cette partie de la formation.

© 2026 iAlexMG. Tous droits réservés.

La partie B s’achève ici, et il ne reste que le capstone. Le Cours 12 reprend un module par étape, du cadrage de la question jusqu’au chargement rejouable, et sa dernière étape est précisément celle que vous venez de faire. Un lecteur qui aurait sauté la partie B s’en apercevrait là.

Prochaine étape : les exercices, dans `Exercices_corriges_11_SQL.md`.

© 2026 iAlexMG. Tous droits réservés.